

Docker & Docker Compose

Week 4 Report • DevOps Internship • Davine Technologies

This report documents my hands-on work with Docker and Docker Compose during Week 4 of the DevOps internship. I installed Docker Engine, built a custom multi-container application (frontend, backend, and database), configured persistent storage and a custom network, and published my images to Docker Hub. Each section below explains a core Docker concept and how I applied it in this week's task.

1. What is Docker?

Docker is a containerization platform that packages an application together with all of its dependencies, libraries, and configuration into a single portable unit called a **container**. Unlike a virtual machine, a container shares the host machine's operating system kernel, which makes it lightweight, fast to start, and consistent across environments — the same container runs identically on my laptop, a teammate's machine, or a production server.

2. Docker Architecture

Docker follows a client-server architecture with three main components:

- **Docker Client** — the CLI (docker command) I use to issue commands.
- **Docker Daemon (dockerd)** — a background service that builds, runs, and manages images, containers, networks, and volumes.
- **Docker Registry (Docker Hub)** — a remote store for images that the daemon pulls from and pushes to.

When I run `docker run`, the client sends the request to the daemon, which pulls the image (if not cached locally) and starts a container from it.

3. Images vs Containers

An **image** is a read-only template built from a Dockerfile — it contains the application code, runtime, and dependencies, but nothing is running yet. A **container** is a running (or stopped) instance of an image, with its own writable layer on top. I can create many containers from the same image, just like I can create many objects from one class. In this task, I built two images — `backend-app` and `frontend` — and ran each as its own container.

4. Dockerfile

A Dockerfile is a plain-text script of instructions Docker uses to build an image layer by layer. Here is the Dockerfile I wrote for my Node.js backend service:

```
FROM node:20-alpine
WORKDIR /app
COPY package.json .
RUN npm install
```

```
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

Each instruction creates a cached layer: FROM sets the base image, WORKDIR sets the working directory, COPY and RUN install dependencies and add source code, EXPOSE documents the container's listening port, and CMD defines the default startup command.

5. Docker Volumes

Volumes provide persistent storage that lives outside a container's writable layer, so data survives even if the container is stopped and removed. I created a named volume and mounted it into my backend container:

```
docker volume create app-data
docker run -d --name backend -p 3000:3000 -v app-data:/app/data backend-app:1.0
```

To verify this, I wrote a test file inside the mounted path, then deleted the container completely and started a brand-new one with the same volume attached. The file was still there — confirming the data is owned by the volume, not the container.

6. Docker Networks

By default, containers are isolated from each other. Docker lets you create a custom **bridge network** so containers can discover and communicate with one another using their container name as a hostname — no manual IP configuration needed. I created a network and confirmed name-based resolution:

```
docker network create app-network
docker run -it --rm --network app-network alpine ping -c 3 backend
```

The ping succeeded, resolving backend to its container IP with 0% packet loss — proving Docker's built-in DNS works on custom networks. This is the same mechanism Docker Compose uses automatically to let my frontend, backend, and database containers talk to each other.

7. Docker Compose

Docker Compose lets me define and run a multi-container application from a single `docker-compose.yml` file, instead of manually running several `docker run` commands. For this task, I deployed a three-service application: a **frontend** (Nginx serving static HTML), a **backend** (Node.js/Express API), and a **database** (MySQL), all connected on one custom network with a named volume for the database's data.

```
services:
  frontend:
    build: ./frontend
    ports:
      - "8080:80"
    networks: [app-network]
    depends_on: [backend]

  backend:
    build: ./backend
```

```

ports:
  - "3000:3000"
environment:
  - DB_HOST=database
volumes:
  - app-data:/app/data
networks: [app-network]
depends_on: [database]

database:
  image: mysql:8
  environment:
    - MYSQL_ROOT_PASSWORD=example
    - MYSQL_DATABASE=appdb
  volumes:
    - db-data:/var/lib/mysql
  networks: [app-network]

networks:
  app-network:
    driver: bridge

volumes:
  app-data:
  db-data:

```

Running `docker compose up -d --build` built both custom images and started all three containers together, each reachable by its service name on the shared network. `docker compose ps` confirmed all three (frontend, backend, database) were up and healthy, and the frontend page loaded successfully in the browser while pinging the backend container by name.

8. Docker Hub

Docker Hub is a cloud registry for storing and distributing Docker images. After building my backend image locally, I tagged and pushed it to my own Docker Hub repository:

```

docker login
docker tag week4-docker-backend:latest srivineeth/backend-app:1.0
docker push srivineeth/backend-app:1.0

```

The image is now publicly available at `hub.docker.com/r/srivineeth/backend-app`, meaning anyone (or any server) can pull and run it with `docker pull srivineeth/backend-app` without needing my source code.

Summary

- Installed and verified Docker Engine on Ubuntu.

- Pulled images from Docker Hub and managed containers (create, run, stop, remove).
- Built a custom Docker image for a Node.js backend using a Dockerfile.
- Configured a named volume and proved data persists independently of the container.
- Created a custom bridge network and confirmed container-to-container DNS resolution.
- Deployed a 3-tier application (frontend + backend + database) with Docker Compose.
- Pushed a custom image to my own Docker Hub repository.